



PROYECTO DE APP BURSATIL

PROYECTO APP DE BOLSA COTARELO



31 DE DICIEMBRE DE 2025

ALUMNOS: IVÁN DURÁN, SAÚL FERNANDEZ, MARIO RODRIGUEZ
PSP

1. INTRODUCCIÓN	1
2. Diseño de arquitectura:	4
2.1. Estructura por capas	4
2.2. Flujo de datos y comunicación entre capas	4
2.3. Concurrencia y ejecución en paralelo	5
2.4. Arquitectura multiplataforma	5
3. Descripción de Funcionalidades	6
3.1. Simulación del mercado bursátil	6
3.2. Visualización de acciones y evolución de precios	7
3.3. Gestión del portafolio de usuario	8
3.4. Sistema de alertas de precio	9
3.5. Simulación del mercado bursátil	10
4. MANUAL DE USUARIO	11
4.1. Pantalla de Inicio	11
4.2. Pantalla de Portafolio	11
4.3. Pantalla de Gráficos	12
4.4. Pantalla de alertas	13
4.5. Estadísticas	13
5. Pruebas	15
5.1. Introducción	15
5.2. Pruebas implementadas	15
5.3. Conclusiones de las pruebas	17
6. CONCLUSIONES Y DIFICULTADES	18
7. REFERENCIAS	19
8. Anexo: Herramientas IA	20

8.1.	Conversación 1 – Creación inicial del proyecto y estructura de carpetas	20
8.2.	Conversación 2 – Motor de simulación y concurrencia _____	21
8.3.	Conversación 3 – Desarrollo del portfolio y lógica de negocio _____	21
8.4.	Conversación 4 – Persistencia del estado mediante JSON_____	22
8.5.	Conversación 5 – Ajustes y validaciones durante el desarrollo _____	22
8.6.	Conversación 6 – Implementación de tests y cierre del proyecto _____	23
9.	<i>Distribución de trabajo Grupal</i> _____	24
10.	<i>ANEXO REPOSITORIO</i> _____	25

1. INTRODUCCIÓN

El presente proyecto, titulado **BolsaCotarelo – Simulador de Mercado Bursátil en Tiempo Real**, consiste en el desarrollo de una aplicación multiplataforma cuyo objetivo principal es simular el funcionamiento básico de un mercado de valores. La aplicación permite representar la evolución de diferentes acciones ficticias, cuyos precios varían de forma dinámica, ofreciendo al usuario la posibilidad de interactuar con el mercado mediante la compra y venta de acciones, la gestión de un portfolio personal y la configuración de alertas de precio.

El proyecto ha sido desarrollado como parte del módulo **Programación de Servicios y Procesos (PSP)** del ciclo formativo de **Desarrollo de Aplicaciones Multiplataforma (DAM)**, poniendo especial énfasis en el uso de la **conurrencia** y la **programación asíncrona**. Para ello, el sistema simula la actualización de precios en tiempo real utilizando hilos y corrutinas, permitiendo que múltiples acciones se actualicen de manera independiente y simultánea, de forma similar a un entorno bursátil real.

La aplicación ha sido implementada utilizando **Kotlin Multiplatform**, lo que permite su ejecución tanto en dispositivos **Android** como en **entornos de escritorio**, compartiendo la mayor parte de la lógica de negocio entre plataformas. La interfaz gráfica ha sido desarrollada con **Jetpack Compose**, adoptando un enfoque declarativo y reactivo que facilita la actualización automática de la interfaz en función de los cambios en el estado de la aplicación.

Entre los requisitos generales del proyecto se incluyen la simulación de precios de mercado en tiempo real, la correcta gestión del estado de las acciones y del portfolio del usuario, la implementación de un sistema de alertas configurable y el uso de una arquitectura organizada y modular. Todo ello se ha diseñado siguiendo buenas prácticas de programación, con el objetivo de obtener una aplicación mantenible, escalable y fácilmente extensible en futuras mejoras.

Para el desarrollo del proyecto **BolsaCotarelo – Simulador de Mercado Bursátil en Tiempo Real** se han utilizado diversas tecnologías y herramientas orientadas a la creación de aplicaciones multiplataforma modernas, haciendo especial énfasis en la concurrencia, la reutilización de código y la separación de responsabilidades.

Kotlin Multiplatform

El proyecto ha sido desarrollado utilizando **Kotlin Multiplatform (KMP)**, una tecnología que permite compartir la lógica de negocio entre diferentes plataformas. Gracias a este enfoque, la mayor parte del código se encuentra en el módulo común (commonMain), mientras que las partes específicas de cada plataforma se ubican en los módulos correspondientes a **Android** y **Escritorio**. Esta estrategia reduce la duplicación de código y facilita el mantenimiento del sistema.

Jetpack Compose y Compose Multiplatform

La interfaz gráfica de la aplicación ha sido implementada mediante **Jetpack Compose** y **Compose Multiplatform**, utilizando un enfoque declarativo para la construcción de la UI. Este modelo permite que la interfaz se actualice automáticamente en función de los cambios de estado, mejorando la claridad del código y la experiencia de usuario. Además, el uso de Compose facilita la reutilización de componentes visuales entre plataformas.

Kotlin Coroutines

Para la simulación del mercado bursátil en tiempo real se ha empleado el uso de **corrutinas de Kotlin**, lo que permite ejecutar tareas concurrentes de forma eficiente y segura. Cada acción del mercado se actualiza de manera independiente, simulando un entorno realista en el que múltiples precios cambian simultáneamente sin bloquear la interfaz de usuario.

Gradle con Kotlin DSL

La gestión del proyecto y sus dependencias se ha realizado mediante **Gradle**, utilizando **Kotlin DSL** para la configuración de los archivos de construcción. Esta elección proporciona una mayor claridad en la definición de módulos, dependencias y configuraciones específicas de cada plataforma, además de una mejor integración con Kotlin Multiplatform.

Android SDK

Para la versión móvil de la aplicación se ha utilizado el **Android SDK**, permitiendo ejecutar la aplicación en dispositivos Android compatibles. Se han integrado los componentes necesarios para la correcta ejecución de la interfaz, la gestión del ciclo de vida y la compatibilidad con las versiones actuales del sistema operativo.

Serialización de datos

El proyecto hace uso de **kotlinx.serialization** para la gestión y persistencia de datos en formato **JSON**, facilitando el almacenamiento y la recuperación de información relacionada con el estado del portfolio y otras estructuras del sistema. Esta tecnología permite una serialización segura y multiplataforma, coherente con el enfoque de Kotlin Multiplatform.

2. Diseño de arquitectura:

La aplicación **BolsaCotarelo – Simulador de Mercado Bursátil en Tiempo Real** ha sido diseñada siguiendo una arquitectura modular y desacoplada, con el objetivo de facilitar el mantenimiento, la escalabilidad y la reutilización del código en un entorno multiplataforma. Para ello, se ha adoptado un enfoque basado en la separación por capas, inspirado en los principios de la **arquitectura limpia (Clean Architecture)** y el patrón **MVVM**.

2.1. Estructura por capas

El sistema se organiza en distintas capas, cada una con responsabilidades claramente definidas:

- **Capa de presentación (Presentation)**
Es la encargada de la interfaz de usuario y la interacción con el usuario final. Está implementada mediante **Jetpack Compose / Compose Multiplatform**, utilizando componentes declarativos que reaccionan automáticamente a los cambios de estado. Esta capa no contiene lógica de negocio, sino que delega dicha responsabilidad en las capas inferiores.
- **Capa de dominio (Domain)**
Contiene la lógica de negocio principal de la aplicación. En esta capa se definen los modelos fundamentales del sistema, como las acciones, el portfolio del usuario y las reglas de alertas. Asimismo, se incluyen las operaciones que gobiernan el comportamiento del mercado simulado, manteniendo la independencia respecto a la plataforma o la interfaz gráfica.
- **Capa de datos (Data)**
Es responsable de la gestión de la simulación del mercado y de la persistencia de la información. En esta capa se implementan los repositorios y servicios que proporcionan los datos necesarios a la capa de dominio, así como los mecanismos de almacenamiento y recuperación del estado de la aplicación.

Esta separación permite que cada capa evolucione de forma independiente, reduciendo el acoplamiento y mejorando la claridad del diseño.

2.2. Flujo de datos y comunicación entre capas

La comunicación entre capas sigue un flujo unidireccional, en el que la interfaz de usuario solicita información a través de modelos intermedios y recibe los cambios de estado de forma reactiva. La lógica de negocio no depende directamente de la interfaz, lo que facilita la reutilización del código común en diferentes plataformas.

El estado de la aplicación se gestiona mediante flujos reactivos, lo que permite que cualquier cambio en los datos del mercado, el portfolio o las alertas se refleje automáticamente en la interfaz gráfica sin necesidad de intervención manual.

2.3. Concurrencia y ejecución en paralelo

Uno de los aspectos clave del diseño de la arquitectura es la gestión de la concurrencia. La simulación del mercado bursátil se basa en la ejecución simultánea de múltiples tareas, donde cada acción puede actualizar su precio de manera independiente.

Para ello, se utilizan **corrutinas**, que permiten ejecutar procesos concurrentes de forma eficiente y segura, evitando bloqueos en la interfaz de usuario. Este enfoque es especialmente adecuado para simular entornos en tiempo real y cumple con los requisitos del módulo de Programación de Servicios y Procesos.

2.4. Arquitectura multiplataforma

Gracias al uso de **Kotlin Multiplatform**, la arquitectura del sistema permite compartir la mayor parte de la lógica entre las versiones Android y Escritorio. La capa común contiene la lógica de negocio y la simulación del mercado, mientras que las capas específicas de cada plataforma se encargan de la integración con el sistema operativo correspondiente.

Este diseño reduce la duplicación de código y garantiza un comportamiento consistente de la aplicación en todas las plataformas soportadas.

3. Descripción de Funcionalidades

3.1. Simulación del mercado bursátil

La aplicación implementa un motor de simulación que genera de forma continua la evolución de los precios de distintas acciones ficticias. Cada acción dispone de un precio inicial y se actualiza de manera periódica, aplicando variaciones aleatorias que simulan el comportamiento real de los mercados financieros.

Estas actualizaciones se ejecutan de forma concurrente mediante el uso de corrutinas, permitiendo que múltiples acciones modifiquen su precio de manera independiente y simultánea sin afectar al rendimiento general de la aplicación. De esta forma, el usuario puede observar cambios de precios en tiempo real, lo que aporta dinamismo y realismo a la simulación.



Ilustración 1: Pantalla Principal de la App

3.2. Visualización de acciones y evolución de precios

La interfaz principal de la aplicación muestra un listado de las acciones disponibles en el mercado. Para cada acción se presenta información relevante como el nombre, el precio actual y su variación respecto al valor anterior.

La aplicación utiliza elementos visuales como colores o indicadores para reflejar si el precio de una acción ha aumentado o disminuido, facilitando la interpretación de la información por parte del usuario. La interfaz se actualiza automáticamente conforme se producen cambios en los datos, gracias al modelo reactivo implementado con Jetpack Compose.

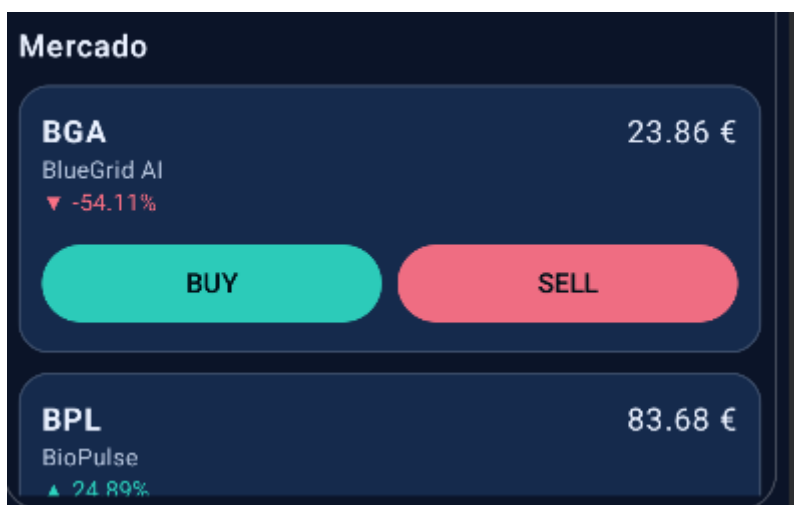


Ilustración 2: Captura de la pantalla Mercado

3.3. Gestión del portfolio de usuario

La aplicación permite al usuario gestionar un **portfolio personal**, desde el cual puede realizar operaciones de compra y venta de acciones. Al comprar una acción, esta se añade al portfolio junto con la cantidad adquirida y el precio de compra, mientras que la venta permite reducir o eliminar posiciones existentes.

El sistema calcula de forma automática el valor total del portfolio, así como las ganancias o pérdidas derivadas de la variación de los precios del mercado. Esta funcionalidad permite al usuario evaluar el impacto de sus decisiones de inversión dentro del entorno simulado.



Ilustración 3: Pantalla del Porfolio

3.4. Sistema de alertas de precio

BolsaCotarelo incorpora un sistema de alertas que permite al usuario definir reglas personalizadas basadas en el precio de las acciones. El usuario puede configurar alertas que se activan cuando una acción supera o desciende por debajo de un valor determinado.

El sistema evalúa continuamente las condiciones de las alertas en paralelo a la simulación del mercado. Cuando se cumple una de estas condiciones, la alerta se dispara y se notifica al usuario dentro de la aplicación, permitiéndole reaccionar ante cambios relevantes del mercado.

Nueva alerta

Ticker

BGA	BPL	CPN	ECW	FRC
HLP	IBK	MDC	MGM	NBS
NDO	QAP	UBT	UCR	VTC

Tipo

Precio $\geq$	Precio $\leq$	% $\geq$	% $\leq$
---------------	---------------	----------	----------

Umbral (ej: 105.5 ó 0.8)

100

Cancelar Crear

Ilustración 4: Menú de creación de alertas

3.5. Simulación del mercado bursátil

La aplicación incluye mecanismos para la persistencia de datos, lo que permite conservar información relevante entre ejecuciones, como el estado del portfolio o las configuraciones de alertas. Los datos se almacenan utilizando formatos de serialización compatibles con Kotlin Multiplatform, garantizando su correcta recuperación tanto en la versión Android como en la versión de escritorio.

Gracias a esta funcionalidad, el usuario puede cerrar y volver a abrir la aplicación sin perder el estado de su simulación ni las operaciones realizadas previamente.

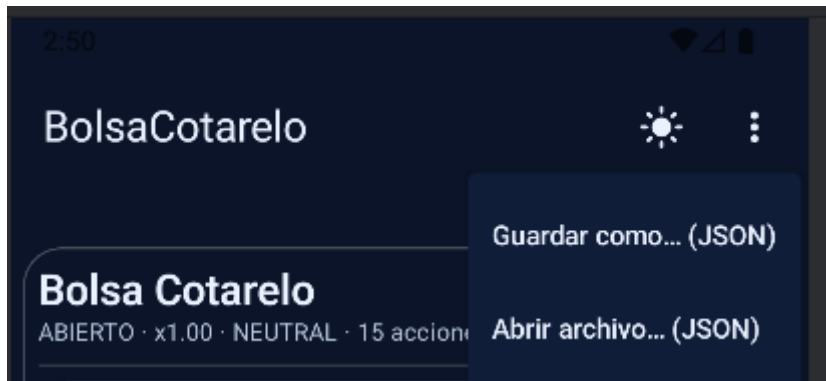


Ilustración 5: Menú superior de Guardado

4. MANUAL DE USUARIO

4.1. Pantalla de Inicio

1. Al iniciar la aplicación se mostrará la pantalla principal del mercado, que contiene:

- Tabla de cotizaciones con las acciones disponibles.
- Columnas con información relevante:
 - Símbolo de la acción.
 - Precio actual.
 - Variación en euros (€).
 - Variación porcentual (%).
- Indicadores visuales de subida o bajada mediante colores y símbolos.
- Panel superior con:
 - Saldo disponible.
 - Valor total del portfolio.
 - Beneficio o pérdida total (PnL).
 - Estado del mercado (ABIERTO / PAUSADO / CERRADO).
- Botones de control del mercado:
 - Cerrar mercado.
 - Pausar mercado.
 - Gestión del horario del mercado.
- Selector de velocidad del mercado (x1.00, 0.5, 1, 2, 5).
- Panel de noticias y sectores en movimiento, que muestra eventos que afectan al mercado y a sectores concretos.
- Información adicional por acción, como precio de apertura, máximo y mínimo.
- Botón para ocultar o mostrar información secundaria.

2. El usuario puede:

- Seleccionar una acción desde la tabla de cotizaciones.
- Comprar o vender acciones mediante los botones BUY y SELL.
- Visualizar cambios de precio en tiempo real.
- Cambiar entre las distintas pestañas de la aplicación mediante la navegación lateral.
- Consultar noticias y movimientos sectoriales que influyen en el mercado.
- Identificar rápidamente acciones en alza o en caída mediante colores e iconos.

3. El mercado:

- Se actualiza automáticamente.
- Los precios cambian en tiempo real.
- Las variaciones se reflejan instantáneamente en la tabla.
- El usuario no necesita refrescar manualmente la información

4.2. Pantalla de Portfolio

1. La pantalla de Portfolio muestra las inversiones realizadas por el usuario y su estado actual, que tiene:
 - Lista de posiciones abiertas con:
 - Acción adquirida.
 - Cantidad de acciones.
 - Precio medio de compra.
 - Precio actual de mercado.
 - Valor total de la posición.
 - Beneficio o pérdida (€ y %).
 - Mensaje informativo cuando no existen posiciones abiertas.
 - Panel de resumen con:
 - Saldo disponible.
 - Capital invertido.
 - Valor total del portfolio.
 - Sección de transacciones con el historial de operaciones realizadas:
 - ID de transacción.
 - Tipo de operación (BUY / SELL).
 - Acción.
 - Cantidad.
 - Fecha y hora.
 - Importe bruto, comisión y total neto.
 - Botones de gestión:
 - Estrategias.
 - Export CSV.
2. El usuario puede:
 - Consultar el rendimiento de cada posición.
 - Ordenar las posiciones según beneficio, pérdida o valor.
 - Visualizar el historial completo de compras y ventas.
 - Exportar el historial de transacciones en formato CSV.
 - Acceder a opciones de estrategias automáticas.
3. La información:
 - Se actualiza automáticamente.
 - El valor de las posiciones cambia en tiempo real según el mercado.
 - El beneficio o pérdida se recalcula de forma automática.
 - El historial de transacciones se actualiza al realizar compras o ventas.

4.3. Pantalla de Gráficos

1. La pantalla de Gráficos permite analizar visualmente la evolución del mercado y del portfolio, en esta pantalla encontramos:

- Selector de acciones (ticker) para elegir la acción a visualizar.
- Gráfico de líneas con la evolución del precio de la acción seleccionada.
- Información de la acción activa:
 - Ticker seleccionado.
 - Precio actual.
 - Variación en euros y porcentaje.
- Gráfico de distribución del portfolio por sector (gráfico de tarta).
- Gráfico de beneficios y pérdidas por acción.
- Ranking de acciones según impacto.
- Filtro para mostrar únicamente las acciones más relevantes.

2. El usuario puede:

- Cambiar la acción visualizada en el gráfico.
- Analizar el comportamiento histórico de precios.
- Consultar la distribución del portfolio por sectores.
- Identificar las acciones con mayor impacto positivo o negativo.

3. Los gráficos:

- Se actualizan dinámicamente.
- Los datos mostrados reflejan el estado actual del mercado.
- Los cambios de precio se representan en tiempo real.
- Los gráficos se adaptan automáticamente al cambiar de acción o filtro.

4.4. Pantalla de alertas

1. La pantalla de Alertas permite gestionar y consultar avisos de mercado, y contiene:

- Lista de alertas configuradas.
- Contadores de alertas activas y disparadas.
- Sección de reglas de alertas.
- Sección de historial de alertas activadas.
- Indicador del estado del mercado.
- Botón OPEN accesible desde la pantalla.

2. El usuario puede:

- Crear nuevas alertas para acciones específicas.
- Configurar condiciones de subida o bajada de precio.

- Consultar alertas ya activadas.
- Eliminar o modificar alertas existentes.

3. Las alertas:

- Funcionan de forma automática.
- El sistema monitoriza continuamente los precios de mercado.
- Las alertas se disparan cuando se cumplen las condiciones configuradas.
- Las alertas activadas se registran en el historial.

4.5. Estadísticas

1. La pantalla de Estadísticas muestra un resumen del rendimiento global del usuario, enseñando:

- Contador de posiciones activas.
- Tarjetas de indicadores principales:
 - Ventas con beneficio.
 - Tasa de éxito.
 - Rentabilidad media.
 - PnL actual.
- Resumen financiero detallado:
 - Saldo disponible.
 - Capital invertido.
 - Valor de las posiciones.
 - Valor total del portfolio.
 - Beneficio o pérdida en porcentaje.
- Sección de mejores y peores resultados:
 - Mejor venta.
 - Peor venta.
 - Acción más rentable.
- Mensaje de ayuda que recomienda el uso de la sección de gráficos para un análisis más detallado.

2. El usuario puede:

- Consultar estadísticas generales del rendimiento.
- Analizar la efectividad de sus operaciones.
- Identificar las acciones más rentables y las menos favorables.

3. Las estadísticas:

- Se recalculan automáticamente.
- Los valores se actualizan tras cada operación realizada.
- Los indicadores reflejan el rendimiento real del usuario.
- No es necesaria ninguna acción manual para actualizar los datos.

5. Pruebas

5.1. Introducción

Con el objetivo de verificar el correcto funcionamiento de la aplicación y demostrar la integración de pruebas automatizadas en un proyecto **Kotlin Multiplatform**, se han implementado una serie de **tests unitarios básicos** centrados en la **lógica de negocio**, la **persistencia de datos** y el **comportamiento concurrente del sistema**, sin depender de la interfaz gráfica.

Las pruebas se han desarrollado utilizando el framework estándar **kotlin.test** y se han ubicado en el `sourceSet commonTest`, garantizando su compatibilidad tanto con la versión Android como con la versión Desktop de la aplicación.

5.2. Pruebas implementadas

Prueba 1: Compra y venta de acciones (PortfolioBuySellTest)

Objetivo:

Verificar que las operaciones de compra y venta de acciones actualizan correctamente el efectivo disponible (*cash*), las posiciones del portfolio (*holdings*) y aplican las comisiones correspondientes.

Descripción:

Se simula una compra de acciones seguida de una venta parcial, comprobando:

- Creación del holding tras la compra.
- Actualización correcta del efectivo.
- Reducción de la cantidad tras la venta.
- Consistencia del estado final del portfolio.

Prueba 2: Control de errores en operaciones inválidas (PortfolioErrorsTest)

Objetivo:

Comprobar que el sistema gestiona correctamente situaciones de error.

Descripción:

Se verifican dos casos:

- Intento de compra con cantidad inválida.
- Intento de venta sin disponer de acciones suficientes.
- En ambos casos, el sistema devuelve errores controlados sin modificar el estado interno.

Prueba 3: Restricción de operaciones según estado del mercado
(PortfolioMarketClosedTest)

Objetivo:

Verificar que las operaciones de trading respetan el estado del mercado.

Descripción:

Se comprueba que no es posible realizar operaciones de compra ni previsualización cuando el mercado está cerrado o en pausa.

Prueba 4: Persistencia del portfolio mediante JSON (PortfolioJsonRoundTripTest)

Objetivo:

Comprobar que el estado del portfolio puede exportarse e importarse correctamente.

Descripción:

Se realiza un ciclo completo de:

- Generación de operaciones.
- Exportación del estado a JSON.
- Limpieza del estado.
- Importación del JSON previamente generado.

Se verifica que el estado final coincide con el original.

Prueba 5: Robustez ante datos JSON inválidos (PortfolioJsonSanitizationTest)

Objetivo:

Verificar que el sistema es robusto ante datos persistidos incorrectos.

Descripción:

Se importa un JSON con datos inconsistentes (cantidades inválidas, tickers incorrectos, transacciones erróneas) y se comprueba que el sistema sanea la información sin provocar errores.

Prueba 6: Comportamiento del motor de simulación (SingleStockPriceUpdaterTest)

Objetivo:

Verificar el funcionamiento básico del motor de simulación de precios.

Descripción:

Se comprueba que:

- El motor emite actualizaciones de precios cuando el mercado está abierto.
- No se generan actualizaciones cuando el mercado está en pausa.

5.3. Conclusiones de las pruebas

Las pruebas implementadas permiten verificar los aspectos fundamentales del sistema sin depender de la interfaz gráfica, asegurando la correcta ejecución de la lógica de negocio, la persistencia de datos y el comportamiento concurrente del motor de simulación.

Estas pruebas demuestran la correcta integración de testing automatizado en un proyecto Kotlin Multiplatform y aportan fiabilidad al desarrollo del sistema.

6. CONCLUSIONES Y DIFICULTADES

El trabajo se distribuyó de forma equilibrada: **Mario** lideró la creación de la base y coordinación del núcleo (estructura, motor y pruebas), **Iván** reforzó la calidad y estabilidad (persistencia/bugs y mejoras funcionales), y **Saúl** impulsó la parte visual y de plataforma (modo claro, Android y mejoras de interfaz). Esta división queda respaldada por los commits y autores registrados en el repositorio.

7. REFERENCIAS

[1] Google LLC, “Jetpack Compose Desktop – documentación oficial”, *Android Developers*. [En línea]. Disponible en:
<https://developer.android.com/compose/documentation>

[2] JetBrains, “Kotlin Documentation”, *KotlinLang.org*. [En línea]. Disponible en:
<https://kotlinlang.org/docs/home.html>

[3] Stack Overflow Inc., “Stack Overflow”, *Stack Overflow*. [En línea]. Disponible en:
<https://stackoverflow.com/>

[4] OpenAI, “ChatGPT – overview”, *ChatGPT / OpenAI*. [En línea]. Disponible en:
<https://openai.com/chatgpt/overview/>

8. Anexo: Herramientas IA

Durante el desarrollo del proyecto se ha utilizado la herramienta de inteligencia artificial **ChatGPT** como apoyo en tareas de análisis, revisión y mejora del código.

Uso realizado

La IA se ha empleado para:

- Análisis de la estructura del proyecto Kotlin Multiplatform.
- Identificación de las partes más críticas y testables del sistema.
- Propuesta de pruebas unitarias coherentes con los objetivos del módulo PSP.
- Corrección de errores de compilación y configuración de tests.
- Redacción y estructuración del apartado de pruebas de la documentación.

Consideraciones

El uso de la herramienta ha sido **supervisado en todo momento**, comprendiendo y validando las soluciones propuestas.

No se ha utilizado la IA para generar automáticamente el proyecto completo ni para sustituir el aprendizaje, sino como apoyo técnico y de refuerzo.

8.1. Conversación 1 – Creación inicial del proyecto y estructura de carpetas

Usuario (Prompt):

Vamos a empezar el proyecto del simulador de mercado bursátil en Kotlin Multiplatform.

Queremos que funcione en Android y Desktop.

¿Cómo deberíamos estructurar el proyecto y las carpetas para separar bien la lógica y la interfaz?

IA (Respuesta):

Para un proyecto Kotlin Multiplatform es recomendable:

- Centralizar la lógica común en `commonMain`.
- Separar la lógica de negocio en paquetes como `domain` y `data`.
- Utilizar repositorios para gestionar el estado del mercado y del portfolio.
- Dejar la interfaz en capas superiores que consuman estos repositorios.

Esta estructura facilita el mantenimiento, el testing y evita dependencias directas entre UI y lógica.

8.2. Conversación 2 – Motor de simulación y concurrencia

Usuario (Prompt):

Tenemos que simular un mercado bursátil en tiempo real, con precios que cambian cada pocos segundos.

Estamos usando corrutinas para actualizar los precios.

¿Este enfoque es correcto o deberíamos usar otra cosa?

IA (Respuesta):

El uso de corrutinas es una muy buena elección.

Permite:

- Simular procesos concurrentes de forma eficiente.
- Actualizar precios sin bloquear la aplicación.
- Controlar fácilmente estados como mercado abierto, cerrado o en pausa.

Desde el punto de vista de PSP, este diseño refleja correctamente la ejecución de procesos concurrentes coordinados.

8.3. Conversación 3 – Desarrollo del portfolio y lógica de negocio

Usuario (Prompt):

Hemos empezado a implementar el portfolio con operaciones de compra y venta.

Queremos controlar comisiones, errores y validar el estado.

¿Cómo podemos organizar esta lógica sin mezclarla con la interfaz?

IA (Respuesta):

La mejor opción es concentrar toda la lógica en un repositorio de portfolio.

Ahí puedes:

- Centralizar las operaciones de compra y venta.
- Validar cantidades, cash disponible y holdings.
- Devolver resultados controlados (éxito o error).

De este modo, la interfaz solo consume el resultado sin conocer los detalles internos.

8.4. Conversación 4 – Persistencia del estado mediante JSON

Usuario (Prompt):

Queremos poder guardar y restaurar el estado del portfolio sin usar base de datos. Estamos pensando en exportarlo a JSON. ¿Es suficiente para este proyecto?

IA (Respuesta):

Sí, es totalmente suficiente.

La persistencia mediante JSON permite:

- Guardar el estado completo del portfolio.
- Restaurarlo tras cerrar la aplicación.
- Mantener el proyecto multiplataforma y sencillo.

Además, puedes añadir validaciones para evitar datos corruptos, lo que mejora la robustez del sistema.

8.5. Conversación 5 – Ajustes y validaciones durante el desarrollo

Usuario (Prompt):

Al avanzar con el proyecto hemos tenido que añadir validaciones extra, como impedir operaciones con el mercado cerrado o en pausa. ¿Tiene sentido añadir estas comprobaciones en los repositorios?

IA (Respuesta):

Sí, es el lugar adecuado.

Los repositorios actúan como punto central de acceso al estado compartido, por lo que:

- Son responsables de validar el estado del mercado.
- Evitan operaciones inválidas antes de ejecutarse.

Esto mejora la coherencia del sistema y evita errores difíciles de detectar en entornos concurrentes.

8.6. Conversación 6 – Implementación de tests y cierre del proyecto

Usuario (Prompt):

Ya tenemos el proyecto funcionando.

Ahora necesitamos implementar algunos tests para demostrar que sabemos integrarlos, pero sin cubrir todo el proyecto.

¿Qué pruebas serían las más adecuadas?

IA (Respuesta):

No es necesario buscar una cobertura total.

Es suficiente con implementar:

- Tests de compra y venta del portfolio.
- Tests de control de errores.
- Tests de exportación e importación del estado en JSON.
- Un test básico del motor de simulación.

Ubicándolos en commonTest y usando kotlin.test, demuestras integración de testing, validación de lógica y comprensión de los conceptos de PSP.

9. Distribución de trabajo Grupal

9.1. Iván - Persistencia/JSON, correcciones y parte de UI/gráficos

Iván realizó gran parte del trabajo de **mejora y corrección**, especialmente relacionado con **persistencia y guardado en JSON**, además de aportaciones al motor y a la parte visual/documental. Se refleja en commits como “Persistencia mejorada. Opciones de guardado incluidas”, “BUGS arreglados en Alertas y Guardados de JSON”, además de commits de “Notificaciones arregladas”, “Gráficos añadidos”, “Clases de Presentación”, “README modificado”, y otros ajustes de limpieza.

9.2. Saúl – UI/tema claro, Android y funcionalidades auxiliares

Saúl centró su aportación en la parte de **interfaz y experiencia visual**, especialmente el **modo claro**, además de cambios relacionados con **Android** y funcionalidades auxiliares. Se ve en commits como “Modo Claro creado y notificaciones mejoradas”, “Botones modo claro arreglados”, “Android main y recursos”, y también aportes a componentes del proyecto (“Clases repository subidas”, “Clases Strategy añadidas”).

9.3. Mario – Coordinación, base del proyecto, persistencia y pruebas

Mario se encargó principalmente de **iniciar y estructurar el proyecto**, crear los **modelos base** y avanzar en partes clave del motor, además de tareas de **persistencia y testing**. Esto se observa en commits como el arranque del repositorio (“bootstrap project structure”, “estructura”, “Inicio proyecto”), creación de modelos (“Modelos”), avance del motor (“add market simulation engine”) y ajustes de persistencia (“Persistencia a medias”, “Persistencia si se cierra...”) y finalmente un commit dedicado a “Test”.

10. ANEXO REPOSITORIO

<https://github.com/Rguez00/BolsaCotarelo.git>